

Name: Rob Hartley_____

Place all written answers from `problemset-02.md` here for easier grading.

1. Prove that $\log n! \in \Theta(n \log n)$.

Upper Bound

Every factor in $n! \leq n$

$$n! = (1 \cdot 2 \cdot 3 \dots n)$$

So:

$$n! \leq n^n$$

$$\log n! \leq \log(n^n)$$

$$\log n! \leq n \log n$$

$$\log n! \in O(n \log n)$$

Lower Bound

Suppose n is even, and we look at the greater half of the factorial, like this:

$\frac{n}{2} + 1, \frac{n}{2} + 2, \frac{n}{2} + 3, \dots, n$ There would be $\frac{n}{2}$ of these factors, each of which is at least $\frac{n}{2}$

Therefore we know that $n! \geq (\frac{n}{2})^{n/2}$

So:

$$\log n! \geq \log (\frac{n}{2})^{n/2}$$

$$\log n! \geq \frac{n}{2} \cdot \log(\frac{n}{2})$$

$$\log n! \geq (\frac{1}{2} \cdot n) \cdot \log(\frac{n}{2})$$

$$\log n! \geq (\frac{1}{2} \cdot n) \cdot (\log n - \log 2)$$

$$\log n! \geq (\frac{1}{2} \cdot n) \cdot (\log n) \cdot (1 - \frac{\log 2}{\log n})$$

$$\frac{\log 2}{\log n} \rightarrow 0 \text{ as } n \rightarrow \infty$$

$$\log n! \geq (\frac{1}{2} \cdot n) \cdot (\log n) \cdot (1 - 0)$$

$$\log n! \geq (\frac{1}{2}) \cdot (n \cdot (\log n))$$

$$\text{and so } \log n! \in \Omega(n \log n)$$

Answer

$$\$ \boxed{\begin{array}{l} \text{Since } \log n! \in O(n \log n) \text{ and } \log n! \in \Omega(n \log n) \\ \log n! \in \Theta(n \log n) \end{array}} \$$$

2. (18 pts, 2pts ea.) **Recurrences**

- $T(n) = 2T(n/6) + 1$

$$T(\frac{n}{6}) = 2T(\frac{n}{36}) + 1$$

Level 1

$$T(n) = 2 \cdot (2T(\frac{n}{36}) + 1) + 1$$

$$T(n) = 4T(\frac{n}{36}) + 3$$

$$T(\frac{n}{36}) = 2T(\frac{n}{216}) + 1$$

Level 2

$$T(n) = 4 \cdot (2T(\frac{n}{216}) + 1) + 3$$

$$T(n) = 8T(\frac{n}{216}) + 7$$

Generalized

$$T(n) = 2^k T(\frac{n}{6^k}) + (2^k - 1)$$

Recursion Depth

$$\frac{n}{6^k} = 1$$

$$n = 6^k$$

$$\log_6 n = k$$

Substitute

$$T(n) = 2^{\log_6 n} T\left(\frac{n}{6^{\log_6 n}}\right) + (2^{\log_6 n} - 1)$$

$$T(n) = n^{\log_6 2} T(1) + n^{\log_6 2} - 1$$

$$\boxed{T(n) \in O(n^{\log_6 2})}$$

- $T(n) = 6T(n/4) + n$

$$T\left(\frac{n}{4}\right) = 6T\left(\frac{n}{16}\right) + \frac{n}{4}$$

Level 1

$$T(n) = 6 \cdot \left(6T\left(\frac{n}{16}\right) + \frac{n}{4}\right) + n$$

$$T(n) = 36T\left(\frac{n}{16}\right) + n + \frac{6n}{4}$$

$$T\left(\frac{n}{16}\right) = 6T\left(\frac{n}{64}\right) + \frac{n}{16}$$

Level 2

$$T(n) = 36 \cdot \left(6T\left(\frac{n}{64}\right) + \frac{n}{16}\right) + n + \frac{6n}{4}$$

$$T(n) = 216T\left(\frac{n}{64}\right) + n + \frac{6n}{4} + \frac{36n}{16}$$

Generalized Equation

$$T(n) = 6^k T\left(\frac{n}{4^k}\right) + n \cdot \sum_{i=0}^{k-1} \left(\frac{6}{4}\right)^i$$

Geometric Series

$$\alpha > 1 \text{ so } \sum_{i=0}^n \alpha^i \leq \frac{\alpha}{\alpha-1} \cdot \alpha^n$$

$$\sum_{i=0}^{k-1} \left(\frac{6}{4}\right)^i \leq \left(\frac{\frac{6}{4}}{\frac{6}{4}-1}\right) \cdot \left(\frac{6}{4}\right)^{k-1}$$

$$\sum_{i=0}^{k-1} \left(\frac{6}{4}\right)^i \leq 3 \cdot \left(\frac{6}{4}\right)^{k-1}$$

Recursion Depth

$$\frac{n}{4^k} = 1$$

$$n = 4^k$$

$$k = \log_4 n$$

Substitute into Generalized Equation

$$T(n) \leq 6^k T\left(\frac{n}{4^k}\right) + 3n \cdot \left(\frac{6}{4}\right)^{k-1}$$

$$T(n) \leq 6^{\log_4 n} T\left(\frac{n}{4^{\log_4 n}}\right) + 3n \cdot \left(\frac{6}{4}\right)^{\log_4 n - 1}$$

$$T(n) \leq n^{\log_4 6} T(1) + 3n \cdot \frac{\left(\frac{6}{4}\right)^{\log_4 n}}{\frac{6}{4}}$$

$$T(n) \leq n^{\log_4 6} T(1) + 2n \cdot n^{\log_4 \frac{6}{4}}$$

$$T(n) \leq n^{\log_4 6} T(1) + 2n^{(1+\log_4 \frac{6}{4})}$$

$$T(n) \leq n^{\log_4 6} T(1) + 2n^{(\log_4 4 + \log_4 \frac{6}{4})}$$

$$T(n) \leq n^{\log_4 6} T(1) + 2n^{(\log_4 (4 \cdot \frac{6}{4}))}$$

$$T(n) \leq n^{\log_4 6} T(1) + 2n^{\log_4 6}$$

$$T(n) \leq (n^{\log_4 6}) \cdot (c + 2)$$

$$\boxed{T(n) \in O(n^{\log_4 6})}$$

- $T(n) = 7T(n/7) + n$

Brick Method

$$T(n) = aT\left(\frac{n}{b}\right) + n^d$$

$$a = 7, b = 7, d = 1$$

$$a = b^d \sim \text{balanced: } \Theta(n^d \log n)$$

$$\boxed{T(n) \in O(n \log n)}$$

- $T(n) = 9T(n/4) + n^2$
 Brick Method
 $T(n) = aT(\frac{n}{b}) + n^d$
 $a = 9, b = 4, d = 2$
 $a < b^d$ — root dominated: $\Theta(n^d)$

$$\boxed{T(n) \in O(n^2)}$$

- $T(n) = 4T(n/2) + n^3$
 $T(\frac{n}{2}) = 4T(\frac{n}{4}) + (\frac{n}{2})^3$
 Level 1
 $T(n) = 4 \cdot (4T(\frac{n}{4}) + (\frac{n}{2})^3) + n^3$
 $T(n) = 16T(\frac{n}{4}) + 4 \cdot (\frac{n}{2})^3 + n^3$
 $T(\frac{n}{4}) = 4T(\frac{n}{8}) + (\frac{n}{4})^3$
 Level 2
 $T(n) = 16 \cdot (4T(\frac{n}{8}) + (\frac{n}{4})^3) + 4 \cdot (\frac{n}{2})^3 + n^3$
 $T(n) = 64T(\frac{n}{8}) + 16 \cdot (\frac{n}{4})^3 + 4 \cdot (\frac{n}{2})^3 + n^3$
 $T(n) = 64T(\frac{n}{8}) + \frac{1}{4}n^3 + \frac{1}{2}n^3 + n^3$
 Generalized Equation
 $T(n) = 4^k T(\frac{n}{2^k}) + n^3 \cdot \sum_{i=0}^{k-1} (\frac{1}{2})^i$
 Geometric Series
 $\alpha < 1$ so $\sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}$
 $\sum_{i=0}^{k-1} (\frac{1}{2})^i < \frac{1}{1-\frac{1}{2}}$
 $\sum_{i=0}^{k-1} (\frac{1}{2})^i < 2$
 Recursion Depth
 $\frac{n}{2^k} = 1$
 $n = 2^k$
 $k = \lg n$

Substitute into Generalized Equation

$$\begin{aligned} T(n) &\leq 4^k T(\frac{n}{2^k}) + n^3 \cdot 2 \\ T(n) &\leq 4^{\lg n} T(\frac{n}{2^{\lg n}}) + 2n^3 \\ T(n) &\leq n^{\lg 4} T(1) + 2n^3 \\ T(n) &\leq cn^2 + 2n^3 \end{aligned}$$

$$\boxed{T(n) \in O(n^3)}$$

- $T(n) = 49T(n/25) + n^{3/2} \log n$
 $T(\frac{n}{25}) = 49T(\frac{n}{25^2}) + (\frac{n}{25})^{3/2} \log(\frac{n}{25})$
 Level 1
 $T(n) = 49 \cdot (49T(\frac{n}{25^2}) + (\frac{n}{25})^{3/2} \log(\frac{n}{25})) + n^{3/2} \log n$
 $T(n) = 49^2 T(\frac{n}{25^2}) + 49(\frac{n}{25})^{3/2} \log(\frac{n}{25}) + n^{3/2} \log n$
 $T(\frac{n}{25^2}) = 49T(\frac{n}{25^3}) + (\frac{n}{25^2})^{3/2} \log(\frac{n}{25^2})$

Level 2

$$T(n) = 49^2 \cdot (49T(\frac{n}{25^3}) + (\frac{n}{25^2})^{3/2} \log(\frac{n}{25^2})) + 49(\frac{n}{25})^{3/2} \log(\frac{n}{25}) + n^{3/2} \log n$$

$$T(n) = 49^3 T(\frac{n}{25^3}) + 49^2 (\frac{n}{25^2})^{3/2} \log(\frac{n}{25^2}) + 49(\frac{n}{25})^{3/2} \log(\frac{n}{25}) + n^{3/2} \log n$$

Generalized Equation

$$T(n) = 49^k T(\frac{n}{25^k}) + \sum_{i=0}^{k-1} 49^i (\frac{n}{25^i})^{3/2} \log(\frac{n}{25^i})$$

Recursion Depth

$$\frac{n}{25^k} = 1$$

$$n = 25^k$$

$$k = \log_{25} n$$

Substitute into Generalized Equation

$$T(n) = 49^{\log_{25} n} T(\frac{n}{25^{\log_{25} n}}) + \sum_{i=0}^{\log_{25} n-1} 49^i (\frac{n}{25^i})^{3/2} \log(\frac{n}{25^i})$$

$$T(n) = n^{\log_{25} 49} T(1) + \sum_{i=0}^{\log_{25} n-1} 49^i (\frac{n^{3/2}}{25^{3i/2}}) \log(\frac{n}{25^i})$$

$$T(n) = n^{\log_{25} 49} T(1) + n^{3/2} \cdot \sum_{i=0}^{\log_{25} n-1} (\frac{49}{\sqrt{25^3}})^i \log(\frac{n}{25^i})$$

Geometric Series

$$\alpha < 1 \text{ so } \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}$$

$$\log \frac{n}{25^i} \leq \log n$$

$$\sum_{i=0}^{\log_{25} n-1} (\frac{49}{\sqrt{25^3}})^i \cdot \log n$$

$$\log n \cdot \sum_{i=0}^{\log_{25} n-1} (\frac{49}{125})^i$$

$$\sum_{i=0}^{\infty} (\frac{49}{125})^i = \frac{1}{1-\frac{49}{125}} = \frac{125}{76} = c$$

$$\log n \cdot c$$

Substitute into Generalized Equation 2

$$T(n) \leq n^{\log_{25} 49} \cdot T(1) + n^{3/2} \cdot \log n \cdot \frac{125}{76}$$

$$T(n) \in O(n^{\log_{25} 49} + n^{3/2} \cdot \log n)$$

$$\log_{25} 49 \approx 1.21 < 3/2$$

$T(n) \in O(n^{3/2} \log n)$

- $T(n) = T(n-1) + 2$

$$T(n-1) = T(n-2) + 2$$

Level 1

$$T(n) = (T(n-2) + 2) + 2$$

$$T(n) = T(n-2) + 4$$

$$T(n-2) = T(n-3) + 2$$

Level 2

$$T(n) = (T(n-3) + 2) + 4$$

$$T(n) = T(n-3) + 6$$

Generalized Equation

$$T(n) = T(n-k) + 2k$$

Recursion Depth

$$n - k = 1$$

$$k = n - 1$$

Substitute into Generalized Equation

$$T(n) = T(n - (n-1)) + 2(n-1)$$

$$T(n) = T(1) + 2n - 2$$

$$\boxed{T(n) \in O(n)}$$

- $T(n) = T(n-1) + n^c$, with $c \geq 1$

$$T(n-1) = T((n-1)-1) + (n-1)^c$$

Level 1

$$T(n) = T((n-1)-1) + (n-1)^c + n^c$$

$$T(n) = T(n-2) + (n-1)^c + n^c$$

$$T(n-2) = T((n-2)-1) + (n-2)^c$$

Level 2

$$T(n) = T((n-2)-1) + (n-2)^c + (n-1)^c + n^c$$

$$T(n) = T(n-3) + (n-2)^c + (n-1)^c + n^c$$

Generalized Equation

$$T(n) = T(n-k) + \sum_{i=0}^{k-1} (n-i)^c$$

Recursion Depth

$$n-k=1$$

$$k=n-1$$

Power Sum

$\sum_{i=0}^n (n-i)^c$ is in the format of:

$$\sum_{i=1}^n i^c = \sum_{i=1}^n i^c \in \Theta(n^{c+1})$$

Substitute into Generalized Equation

$$T(n) = T(n-(n-1)) + \sum_{i=0}^{(n-1)-1} (n-i)^c$$

$$T(n) = T(1) + \Theta(n^{c+1})$$

$$\boxed{T(n) \in O(n^{c+1})}$$

- $T(n) = T(\sqrt{n}) + 1$

$$T(n^{\frac{1}{2}}) = T((n^{\frac{1}{2}})^{\frac{1}{2}}) + 1$$

Level 1

$$T(n) = [T((n^{\frac{1}{2}})^{\frac{1}{2}}) + 1] + 1$$

$$T(n) = T(n^{\frac{1}{4}}) + 1 + 1$$

$$T(n^{\frac{1}{4}}) = T((n^{\frac{1}{4}})^{\frac{1}{2}}) + 1$$

Level 2

$$T(n) = [T((n^{\frac{1}{4}})^{\frac{1}{2}}) + 1] + 1 + 1$$

$$T(n) = T(n^{\frac{1}{8}}) + 1 + 1 + 1$$

Generalized Equation

$$T(n) = T(n^{\frac{1}{2^k}}) + \sum_{i=0}^{k-1} 1$$

$$T(n) = T(n^{\frac{1}{2^k}}) + k$$

Recursion Depth

$$n^{\frac{1}{2^k}} \leq 2$$

$$\lg(n^{\frac{1}{2^k}}) \leq \lg(2)$$

$$\frac{1}{2^k} \cdot \lg n \leq 1$$

$$\lg n \leq 2^k$$

$$\lg \lg n \leq k$$

Substitute into Generalized Equation

$$T(n) = T(n^{\frac{1}{2^{\lg \lg n}}}) + \lg \lg n$$

$$T(n) = T(n^{\frac{1}{\lg n}}) + \lg \lg n$$

$$n = 2^{\lg n}$$

$$T(n) = T((2^{\lg n})^{\frac{1}{\lg n}}) + \lg \lg n$$

$$T(n) = T(2) + \lg \lg n$$

$$T(n) \in O(\lg \lg n)$$

3. (8 pts) **Algorithm Selection**

- Algorithm $\mathcal{A}$ solves problems by dividing them into two subproblems of one fifth of the input size, recursively solving each subproblem, and then combining the solutions in quadratic time.

$$W_A(n) = 2W_A(\frac{n}{5}) + \Theta(n^2)$$

$$a = 2, b = 5, d = 2$$

$$a < b^d$$

$$2 < 5^2 \text{ Root Dominated}$$

$$W_A(n) \in \Theta(n^2)$$

- Algorithm $\mathcal{B}$ solves problems of size n by recursively one subproblems of size $n - 1$ and then combining the solutions in logarithmic time.

$$W_B(n) = W_B(n - 1) + \Theta(\log n)$$

Expand

$$W_B(n) = \log n + \log(n - 1) + \log(n - 2) \dots \log(2)$$

$$\log a + \log b = \log(ab)$$

$$W_B(n) = \log(n!)$$

$$W_B \in \Theta(n \log n)$$

- Algorithm $\mathcal{C}$ solves problems of size n by dividing them into a subproblems of size $n/3$ and a subproblem of size $2n/3$, recursively solving each subproblem, and then combining the solutions in $O(n^{1.1})$ time.

$$W_C(n) = W_C(\frac{n}{3}) + W_C(\frac{2n}{3}) + O(n^{1.1})$$

Level 0

$$O(n^{1.1})$$

Level 1

$$O[(\frac{n}{3})^{1.1} + (\frac{2n}{3})^{1.1}]$$

$$O[n^{1.1} \cdot (\frac{1}{3})^{1.1} + (\frac{2}{3})^{1.1}]$$

$$[(\frac{1}{3})^{1.1} + (\frac{2}{3})^{1.1}] < 1$$

Root Dominated

$$W_C(n) \in O(n^{1.1})$$

What is the work and span of these algorithms? For the span, just assume that it is the same as the work to combine solutions (i.e. the non-recursive quantity). Which algorithm would you choose? Why?

$$W_A(n) \in \Theta(n^2), S_A(n) \in \Theta(n^2)$$

$$W_B \in \Theta(n \log n), S_B \in \Theta(\log n)$$

$$W_C(n) \in O(n^{1.1}), S_C(n) \in O(n^{1.1})$$

Algorithm B grows more slowly than the other two, so I'd choose that one.
 One caveat is that we only know the upper bound for Algorithm C, so the actual work and span for Algorithm C could be lower.
 Based on the information we have, however, Algorithm B is the best choice.

4. (8 pts) **More Algorithm Selection**

- Algorithm $\mathcal{A}$ solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.

$$W_A(n) = 5W\left(\frac{n}{2}\right) + \Theta(n)$$

Master Method

$$W(n) = aW\left(\frac{n}{b}\right) + n^c$$

$$a = 5, b = 2, c = 1$$

$$\log_2 5 > 1$$

$$W_A(n) \in \Theta(n^{\log_2 5})$$

- Algorithm $\mathcal{B}$ solves problems of size n by recursively solving two subproblems of size $n - 1$ and then combining the solutions in constant time.

$$W_B(n) = 2W(n - 1) + \Theta(1)$$

Generalized Equation

$$W_B(n) = 2^k W_B(n - k) + c \cdot \sum_{i=0}^{k-1} 2^i$$

Substitute into Generalized Equation

$$W_B(n) = 2^{n-1} W_B(n - (n - 1)) + c \cdot \sum_{i=0}^{(n-1)-1} 2^i$$

Geometric Series

$$\sum_{i=0}^{n-2} 2^i < \frac{2}{2-1} \cdot 2^{n-2} = 2^{n-1} = O(2^n)$$

Substitute into Generalized Equation

$$W_B(n) = 2^{n-1} W_B(1) + c \cdot O(2^n)$$

$$W_B(1) = \Theta(1)$$

$$W_B(n) \in \Theta(2^n)$$

- Algorithm $\mathcal{C}$ solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $O(n^2)$ time.

$$W_C(n) = 9W_C\left(\frac{n}{3}\right) + O(n^2)$$

Master Method

$$W(n) = aW\left(\frac{n}{b}\right) + n^c$$

$$a = 9, b = 3, c = 2$$

$$\log_3 9 = 2$$

$$W_C(n) \in O(n^2 \log n)$$

What is the work and span of these algorithms? For the span, just assume that it is the same as the work to combine solutions (i.e., the non-recursive quantity). Which algorithm would you choose? Why?

$$W_A(n) \in \Theta(n^{\log_2 5}), S_A(n) \in \Theta(n)$$

$$W_B(n) \in \Theta(2^n), S_B(n) \in \Theta(1)$$

$$W_C(n) \in O(n^2 \log n), S_C(n) \in O(n^2)$$

The work for algorithm C grows more slowly than the other two, so I'd choose that one.

5. (4 pts) **Integer Multiplication Timing Results**

n	quadratic	subquadratic
10^1	0.031	0.024
10^2	0.045	0.170
10^4	0.113	0.156
10^8	0.476	0.478
10^{16}	1.428	1.247
10^{32}	5.849	4.016
10^{64}	23.423	13.286
10^{128}	91.620	41.598
10^{256}	373.355	116.666
10^{512}	1452.304	341.128
10^{1024}	5467.930	1016.944

In the tests, the exponent of the decimal input is doubled at each step. Since the number of bits in 10^k is proportional to k , this approximately doubles the input size in bits at each step.

When the input bit-length doubles, we expect:

$$\frac{W_Q(2n)}{W_Q(n)} = \frac{(2n)^2}{n^2} = 2^2 = 4$$

$$\frac{W_{KO}(2n)}{W_{KO}(n)} = \frac{(2n)^{\log_2 3}}{n^{\log_2 3}} = 2^{\log_2 3} = 3$$

Therefore, we would expect the quadratic multiplication algorithm to take approximately 4 times as long

when the input size doubles, while the Karatsuba-Ofman algorithm should take approximately 3 times as long.

For the larger inputs, this is approximately what we observe in the measured running times.